

Scotland's most picturesque running routes

Reboot Data Analyst assessment · Josephin Shajan

An interactive walkthrough of the analysis. This notebook reads the datasets produced by `alltrails_scotland.py` — it makes no requests to AllTrails, so it runs in a few seconds.

Source	AllTrails trail-running routes, Scotland
Sample	100 routes across 6 regional listings
Review text	421 reviews
Collected	31 July 2026

The question: there is no beauty column on AllTrails, so which measurable signals stand in for how a route looks?

```
In [1]: import os
import pandas as pd
import plotly.express as px
import plotly.graph_objects as go
import plotly.io as pio

pio.renderers.default = "notebook"
pd.set_option("display.max_columns", 40)

CANDIDATES = ["reboot_output/data_processed", "data_processed", "."]
BASE = next((p for p in CANDIDATES
             if os.path.exists(os.path.join(p, "routes_scored.csv"))), None)

if BASE is None:
    from google.colab import files
    print("Upload routes_scored.csv and the summary*.csv files")
    files.upload()
    BASE = "."

def load(name):
    path = os.path.join(BASE, name)
    return pd.read_csv(path) if os.path.exists(path) else None

df = load("routes_scored.csv")
sens = load("summary_sensitivity.csv")
by_diff = load("summary_by_difficulty.csv")
tags = load("summary_tag_counts.csv")

print(f"{len(df)} routes loaded, {int(df['headline_eligible'].sum())} eli
df[df["headline_eligible"]][
    ["name", "picturesque_score", "avg_rating", "num_reviews",
     "photos_per_review", "length_km", "elevation_gain_m"]].reset_index(d
```

100 routes loaded, 82 eligible for ranking

Out [1]:

	name	picturesque_score	avg_rating	num_reviews	photos_per_review	le
0	Glencoe Lochan: Lochan Trail	78.6	4.7	267.0	5.262172	
1	Rhenigdale Post Road	76.5	4.8	23.0	3.391304	1
2	Ben Lomond Mountain Path	74.9	4.7	2330.0	2.156652	
3	Coire na Creiche and the Fairy Pools	73.9	4.6	449.0	11.971047	
4	Ben Macdui, Stob Coire an t- Sneachda, Cairn Go...	70.7	4.8	671.0	4.658718	1

1. The sample at a glance

```
In [2]: summary = pd.DataFrame({
    "Measure": ["Routes", "Eligible for ranking (10+ reviews)",
                "Median length (km)", "Median ascent (m)", "Median rating",
                "Median photos per review",
                "Reviews mentioning scenery", "Reviews mentioning a detractor"],
    "Value": [
        len(df),
        int(df["headline_eligible"].sum()),
        round(df["length_km"].median(), 1),
        round(df["elevation_gain_m"].median()),
        round(df["avg_rating"].median(), 2),
        round(df["photos_per_review"].median(), 2),
        f"{df['scenery_mention_rate'].mean() * 100:.1f}%",
        f"{df['detractor_rate'].mean() * 100:.1f}%",
    ],
})
summary
```

Out [2]:

	Measure	Value
0	Routes	100
1	Eligible for ranking (10+ reviews)	82
2	Median length (km)	8.0
3	Median ascent (m)	284
4	Median rating	4.6
5	Median photos per review	2.18
6	Reviews mentioning scenery	61.6%
7	Reviews mentioning a detractor	18.2%

2. The headline finding

If star ratings already told us which routes were beautiful, this whole exercise would be unnecessary. They don't.

Ratings cluster tightly — nearly everything sits between 4.5 and 4.8 — while photos per visitor range from under one to nearly twelve. Hover any point below.

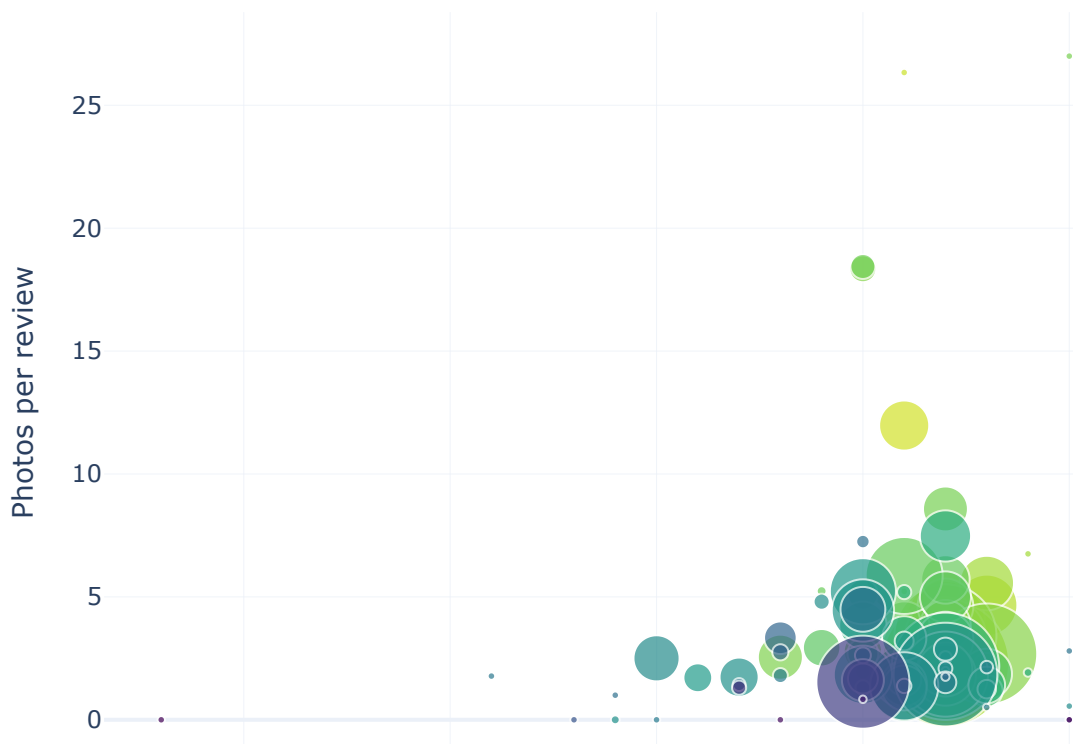
The Fairy Pools is the clearest case: the lowest rating in the top ten, and by far the most photographed route in the sample.

In []:

```
In [3]: d = df.dropna(subset=["photos_per_review", "avg_rating"]).copy()
r = d["photos_per_review"].rank().corr(d["avg_rating"].rank())

fig = px.scatter(
    d, x="avg_rating", y="photos_per_review",
    size=d["num_reviews"].clip(10, 3000), color="picturesque_score",
    hover_name="name",
    hover_data={"num_reviews": ":", "length_km": ":.1f",
                "elevation_gain_m": ":.0f", "avg_rating": ":.2f",
                "photos_per_review": ":.2f", "picturesque_score": ":.1f"},
    color_continuous_scale="Viridis", size_max=45,
    labels={"avg_rating": "Average star rating",
            "photos_per_review": "Photos per review",
            "picturesque_score": "Index"},
    title=(f"Enjoyment and photography measure different things "
           f"(Spearman = {r:.2f})),
)
fig.update_layout(height=560, template="plotly_white")
fig.show()
```

Enjoyment and photography measure different things (Spearman



3. The picturesque index

Five signals, each converted to a percentile rank across the sample, then weighted:

Signal	Weight	What it measures
Photos per review	25%	Did people stop mid-run to photograph it?
Scenery language in reviews	25%	What runners actually talk about
AllTrails view tag	20%	The site's own three-level view rating
Adjusted star rating	15%	Quality control, shrunk toward the mean
Elevation per km	15%	Climbing as a proxy for viewpoints

Percentiles rather than min-max scaling, so one heavily-photographed route can't flatten everything else toward zero.

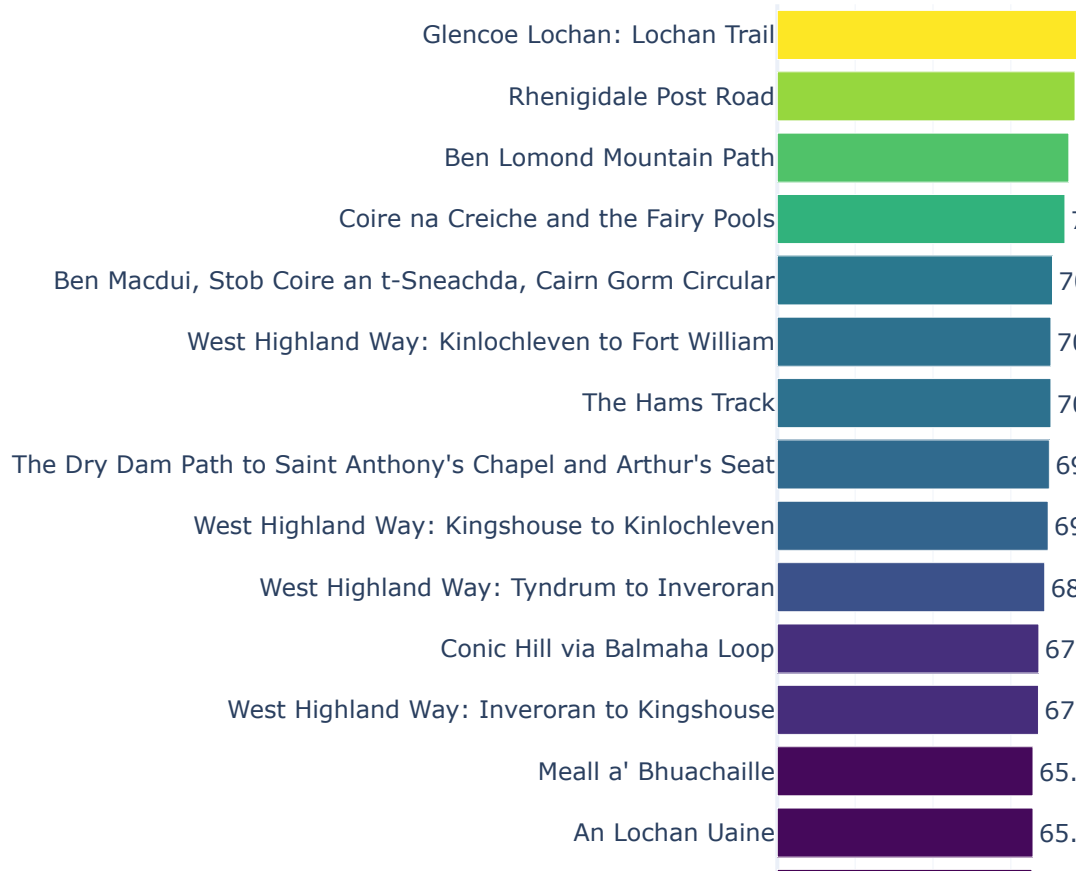
```
In [4]: top = df[df["headline_eligible"]].nlargest(15, "picturesque_score").iloc[
fig = px.bar(
    top, x="picturesque_score", y="name", orientation="h",
    color="picturesque_score", color_continuous_scale="Viridis",
```

```

text=top["picturesque_score"].round(1), hover_name="name",
hover_data={"avg_rating": ":%.1f", "num_reviews": ":",",",
            "photos_per_review": ":%.2f", "name": False,
            "picturesque_score": False},
labels={"picturesque_score": "Picturesque index (0-100)", "name": ""}
title="Scotland's 15 most picturesque running routes",
)
fig.update_traces(textposition="outside")
fig.update_layout(height=640, template="plotly_white",
                  coloraxis_showscale=False, xaxis_range=[0, 92])
fig.show()

```

Scotland's 15 most picturesque running routes



4. You have to earn the view

Ratings are flat across difficulty bands — within 0.08 of each other. Harder routes are not enjoyed more.

Photographs per visitor more than double.

```

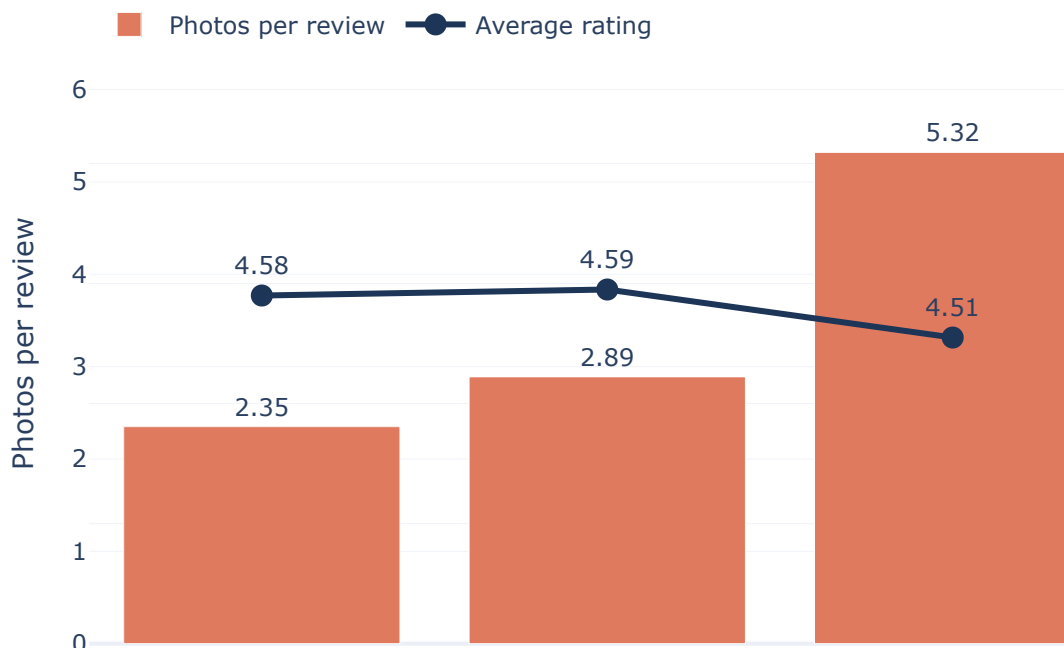
In [5]: order = ["Easy", "Moderate", "Hard"]
bd = by_diff.set_index("difficulty").reindex(order).reset_index()

fig = go.Figure()
fig.add_bar(x=bd["difficulty"], y=bd["mean_photos_per_review"],
            name="Photos per review", marker_color="#e07a5f",
            text=bd["mean_photos_per_review"].round(2), textposition="out
fig.add_scatter(x=bd["difficulty"], y=bd["mean_rating"], yaxi="y2",
               name="Average rating", mode="lines+markers+text",
               line=dict(color="#1d3557", width=3), marker=dict(size=11)
               text=bd["mean_rating"].round(2), textposition="top center
fig.update_layout(
    title="Harder routes are photographed more, but not rated higher",
    yaxi=dict(title="Photos per review", range=[0, 6.5]),
    yaxi2=dict(title="Average rating", overlaying="y", side="right",
               range=[4.0, 5.0]),
    template="plotly_white", height=480,
    legend=dict(orientation="h", y=1.08),
)
fig.show()

bd

```

Harder routes are photographed more, but not rated higher



```

Out [5]:

```

	difficulty	routes	mean_score	mean_rating	mean_photos_per_review	mean_le
0	Easy	26	42.18	4.58	2.35	
1	Moderate	41	52.22	4.59	2.89	
2	Hard	33	52.57	4.51	5.32	

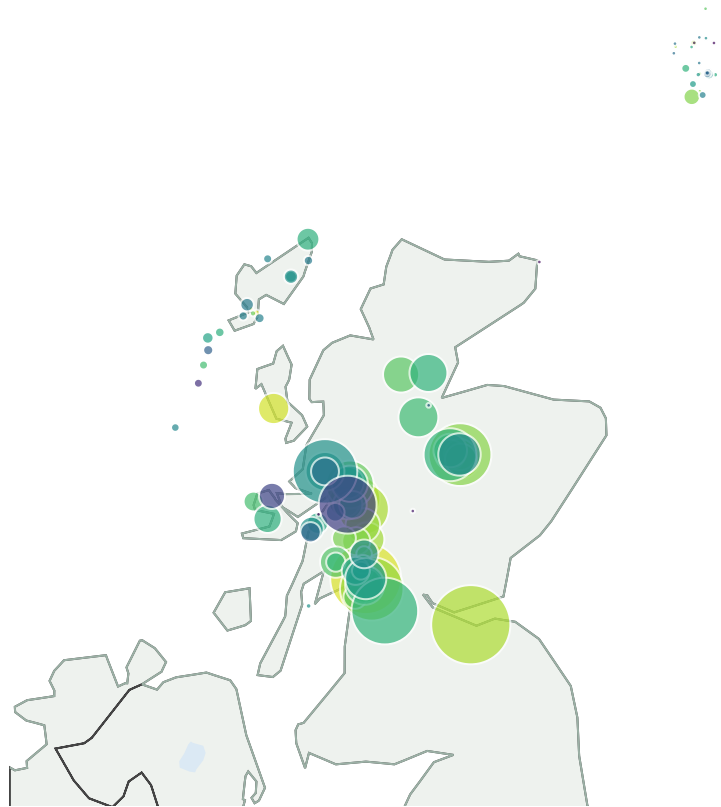
5. Where the high scorers are

Eight of the top ten are Highlands or islands routes. Point size is review volume, colour is the index.

```
In [6]: g = df.dropna(subset=["latitude", "longitude", "picturesque_score"]).copy

fig = px.scatter_geo(
    g, lat="latitude", lon="longitude", color="picturesque_score",
    size=g["num_reviews"].clip(10, 3000), hover_name="name",
    hover_data={"picturesque_score": ":.1f", "avg_rating": ":.1f",
                "photos_per_review": ":.2f", "latitude": False,
                "longitude": False},
    color_continuous_scale="Viridis", size_max=28,
    labels={"picturesque_score": "Index"},
    title="Picturesque index by location",
)
fig.update_geos(scope="europe", resolution=50,
                lataxis_range=[54.5, 61], lonaxis_range=[-9, 0],
                showland=True, landcolor="#eef2ee",
                showlakes=True, lakecolor="#dbe9f4",
                showcoastlines=True, coastlinecolor="#9bb0a5")
fig.update_layout(height=680, template="plotly_white")
fig.show()
```

Picturesque index by location



6. What runners actually write about

Each review is checked against a deliberately narrow scenery word list. "Nice" and "great" describe the run; "stunning" and "viewpoint" describe the view.

"Loch" leading for nearly a quarter of routes is distinctly Scottish — a useful detail for a regional story.

```
In [7]: from collections import Counter

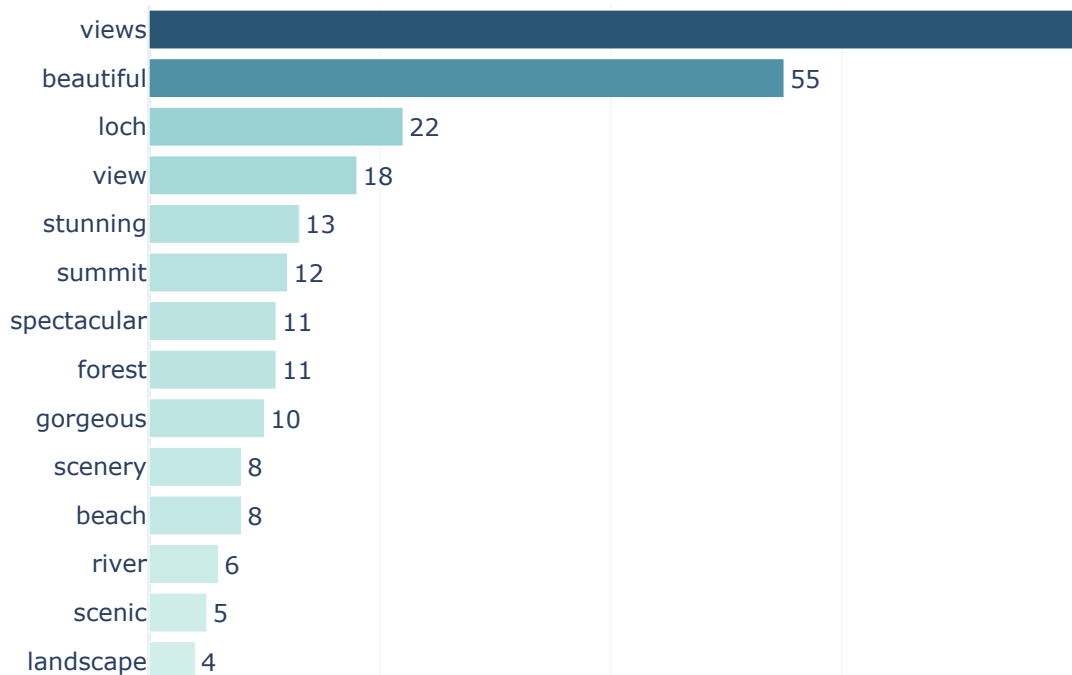
wc = Counter()
for s in df["top_scenery_words"].fillna("").astype(str):
    wc.update(w for w in s.split("|") if w)

words = pd.DataFrame(wc.most_common(14), columns=["word", "routes"])[::-1]
```



```
fig = px.bar(words, x="routes", y="word", orientation="h",
             color="routes", color_continuous_scale="Teal",
             text="routes",
             labels={"routes": "Routes where it is a leading term", "word": "word"},
             title="The words Scotland's runners reach for")
fig.update_traces(textposition="outside")
fig.update_layout(height=520, template="plotly_white",
                  coloraxis_showscale=False)
fig.show()
```

The words Scotland's runners reach for



7. How much to trust the ranking

A ranking that only holds under one exact set of weights isn't a finding. So the index is recomputed under equal weights and five leave-one-out variants, then compared to the chosen version.

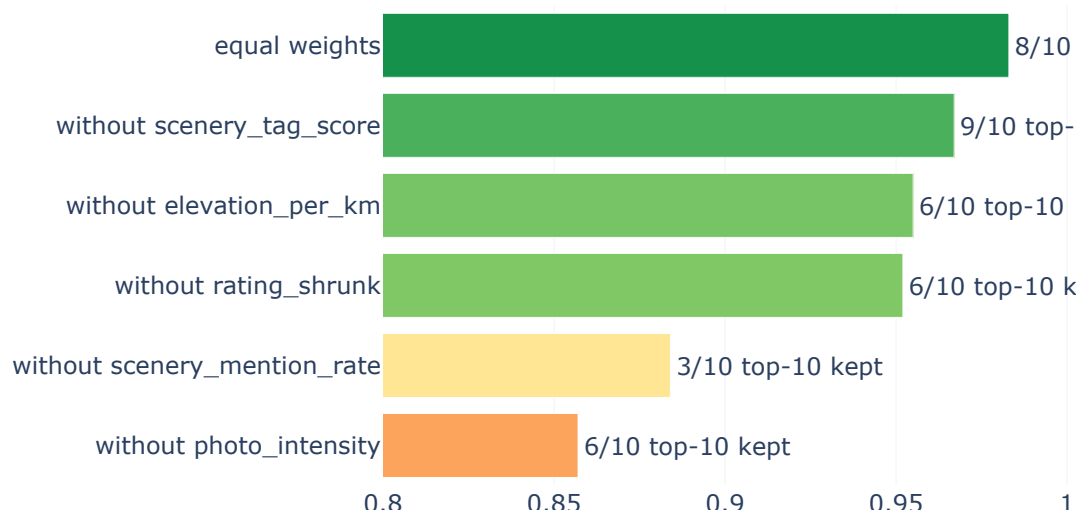
Honest reading: the broad ordering is stable (all variants above 0.85), but exact top-10 membership is not. Dropping the review-language signal changes seven of the ten.

This check also caught a real bug. In an earlier version the view-tag signal carried 30% of the weight and removing it changed the ranking by *nothing* — because it had been built against tags AllTrails doesn't use. A heavy variable making no difference isn't stability, it's a dead signal.

```
In [8]: s = sens.sort_values("spearman_vs_chosen")

fig = px.bar(s, x="spearman_vs_chosen", y="variant", orientation="h",
             text=s["top10_overlap"].astype(str) + "/10 top-10 kept",
             color="spearman_vs_chosen", color_continuous_scale="RdYlGn",
             range_color=[0.8, 1.0], range_x=[0.8, 1.02],
             labels={"spearman_vs_chosen": "Rank correlation with chosen",
                    "variant": ""},
             title="Does the ranking survive different weighting choices?",
             fig.update_traces(textposition="outside")
fig.update_layout(height=420, template="plotly_white",
                  coloraxis_showscale=False)
fig.show()
```

Does the ranking survive different weighting choices?



```
In [9]: # Correlations between the raw signals, not with the composite - a compon
# always correlates with an index it is part of, which proves nothing.
cols = ["avg_rating", "num_reviews", "photos_per_review",
        "scenery_mention_rate", "elevation_per_km", "length_km"]

ranks = df[cols].rank()
corr = ranks.corr().round(3)

fig = px.imshow(corr, text_auto=True, aspect="auto", zmin=-0.5, zmax=1,
                color_continuous_scale="RdBu_r",
                title="Spearman correlation between signals")
fig.update_layout(height=520, template="plotly_white")
fig.show()

print("Rating vs photos per review :",
      corr.loc["avg_rating", "photos_per_review"])
print("Rating vs scenery mentions :",
      corr.loc["avg_rating", "scenery_mention_rate"])
```

Spearman correlation between signals

avg_rating	1	0.084	0.129	-0.035	0.005	0.05
num_reviews	0.084	1	0.266	0.057	0.365	0.002
photos_per_review	0.129	0.266	1	-0.029	0.113	0.331
scenery_mention_rate	-0.035	0.057	-0.029	1	0.218	0.109
elevation_per_km	0.005	0.365	0.113	0.218	1	0.179
length_km	0.05	0.002	0.331	0.109	0.179	1

Rating vs photos per review : 0.129

Rating vs scenery mentions : -0.035

8. Caveats

The index is a proxy, not a measurement of beauty — the sensitivity chart above is the honest answer to how far the exact ordering should be trusted.

The sample is 100 routes, so this is "the most picturesque of these 100", not of Scotland. Roughly ten routes per regional page means the sample isn't proportional to Scotland's route population, so the Highlands' dominance of the top is a genuine result while their share of the sample is a design choice.

Review text is 421 reviews across 100 routes, not every review written. Keyword matching can't distinguish "stunning" from "not stunning at all". Length is missing for 4 routes and elevation for 7, where values failed a plausibility check and were nulled rather than kept.

Full detail in [docs/methodology.md](#) .